

CudaForge: 一种带有硬件反馈的 CUDA 内核最优化智能体框架

Zijian Zhang* Rong Wang* Shiyang Li Yuebo Luo

Mingyi Hong[†] Caiwen Ding[†]

University of Minnesota, Twin Cities

{zha00175, wan00559, li004074, luo00466, mhong, dingc}@umn.edu

Abstract

对于大规模大模型训练等人工智能应用而言，开发高效的 CUDA 内核变得越来越关键。然而，手动设计内核既成本高昂又耗时，这促使人们采用基于大模型的代码生成来实现自动化。但现有的自动内核生成方法往往会产生低效率的内核，带来较高的计算开销，并且在不同情景下无法实现泛化。

在本工作中，我们提出 CudaForge，一种无需训练的多智能体工作流，用于 CUDA 内核生成与优化。我们的工作流受到人类专家迭代式工作流程的启发，包含开发初始内核、测试正确性、分析硬件反馈以及迭代改进等步骤。

更具体地，CudaForge 采用两个大语言模型智能体：一个编码器（Coder）和一个评判器（Judge），它们通过迭代生成、修正与优化 CUDA 内核，并整合如 Nsight Compute (NCU) 指标在内的硬件反馈信息。在广泛的评估中，我们表明，通过利用 OpenAI-o3 等基础模型，CudaForge 实现了 97.6% 的内核生成正确率，并在平均上相比 PyTorch 基准获得 $1.68\times$ 的加速比，在 KernelBench 上显著超越当前最先进模型（包括 OpenAI-o3 和 Kevin）；同时，进一步增加最大迭代轮次可使 CudaForge 的性能提升至 $2.27\times$ 的加速比，展现出其在实际应用中的强大能力。除了准确率与速度外，CudaForge 在多种 GPU（A100、RTX 6000、4090、3090）和基础模型（OpenAI-o3、GPT-5、gpt-oss-120B、Claude-Sonnet-4、QwQ-32B）之间均表现出强大的泛化能力，同时保持高效率。特别地，在一台 RTX6000 上生成一个优化后的内核大约耗时 26.5 分钟，API 成本约为 \$0.3，远低于现有 Agentic 工作流每内核需 6 小时 H100 计算和 \$5 API 成本的水平。我们的结果表明，多智能体、无需训练的工作流能够实现低成本、可泛化且高性能的 CUDA 内核优化。代码已公开于 <https://github.com/OptimAI-Lab/CudaForge>

*Co-first author. Equal contribution; order decided by a coin toss.

[†]Corresponding author.

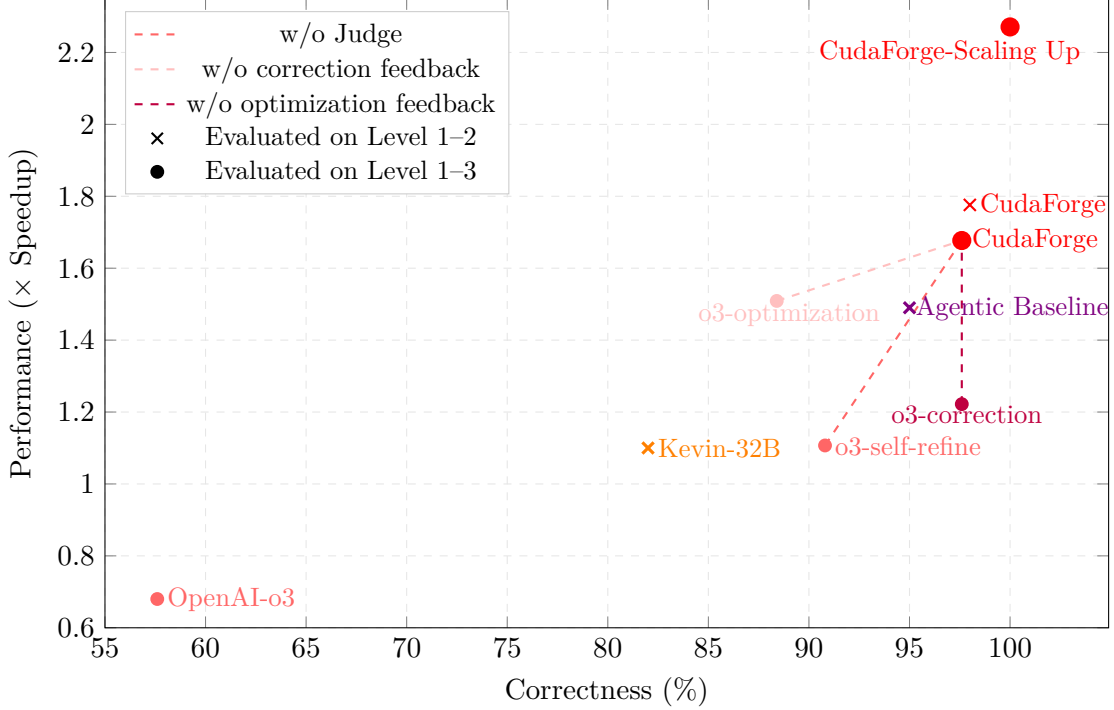


Figure 1: CuDaForge 在 KernelBench 上的正确性和性能均达到了最先进水平，超越了基于强化的方法如 Kevin-32B [?]、Agentic 基准方法 [?] 以及 OpenAI-o3 [?]。为进一步评估我们设计的有效性，我们额外开发了 OpenAI-o3 的三种定制变体：o3-self-refine、o3-correction 和 o3-optimization，作为消融实验的基准。增加最大迭代轮数（CuDaForge-缩放）进一步将 CuDaForge 的性能提升至 2.27 $\times$ 的加速比。实验细节见第 3 节。

1 引言

动机。 CUDA 已经成为深度学习训练的事实标准，因为现代框架如 PyTorch 和 TensorFlow 与 NVIDIA 的优化 GPU 库深度集成 [?]。高效的 CUDA 内核对于加速深度学习工作负载至关重要 [?]。

然而，开发高效的 CUDA 内核被公认为具有挑战性，学习曲线陡峭，需要对 GPU 架构和并行编程有深入的专业知识 [?]。例如，从 Hopper GPU 架构首次发布到 FlashAttentionV3 [?] 的推出，耗时超过两年，而后者是专为 Hopper GPU 设计的。

这一高开发门槛促使人们越来越关注寻找自动化生成高效且定制化 CUDA 内核的方法。例如，一些研究 [?] [?] 采用自动调优和演化搜索来自动探索内核实现空间，并优化特定硬件的底层参数。最近，利用大模型 (LLMs) 完成此类任务的兴趣日益增长。由于大模型在其他领域（如 Python 和 C++）具备代码生成能力，人们相信其在生成高效且高质量的内核方面具有巨大潜力 [?]。

现有工作与关键挑战。 通常情况下，使用大模型生成 CUDA 内核仍处于早期阶段。在 KernelBench [?] 中，作者尝试直接使用最先进的 (SOTA) 模型，如 OpenAI-o1 和 Claude-3.5-Sonnet 来生成内核。然而，已有观察表明，这些 SOTA 模型仍然难以直接生成正确或高性能的内核，揭示

了现有大模型在该领域存在的根本性局限。

为弥补这一空白，近期研究探索了两种主要范式。第一种方法基于强化学习 (RL) [? ?]。CUDA-L1 [?] 和 Kevin [?] 采用强化学习来提升大模型生成正确且高效 CUDA 代码的能力。

第二种方法基于智能体。具体而言，在一项独立且同时进行的研究中 [?] ¹，研究人员在推理时探索了 Agentic 框架。智能体将 PyTorch 方法映射到 CUDA 内核设计，随后通过采样新内核并进行验证过滤来进一步优化 CUDA 内核。该设计在无需高昂的强化学习训练成本的情况下，有效提升了 CUDA 内核生成的正确性。

尽管取得了这些进展，仍存在几个关键挑战：

(C1) 内核效率有限。尽管基于强化学习的方法提升了大模型生成 CUDA 内核的能力，但其最优化能力仍然不足。例如，Kevin-32B 生成的内核在对每个内核采样 16 条并行轨迹并进行 8 次精炼后，仅在 KernelBench Level 1-2 上实现了平均 1.10× 的加速比 [?]。另一个例子是，CUDA-L1 通常无法直接优化 CUDA 内核，但能够生成 PyTorch 的官方实现 [?](see Appendix C for details)。

(C2) 高训练与推理成本。基于强化学习的方法（如 [?]）需要大量的计算资源和漫长的训练周期，使其不适用于低资源或快速原型开发场景。此外，[?] 提出的多阶段 Agentic 流水线在推理时产生高昂的成本（每个核约需 6 小时 H100 计算时间和 5 美元 API 费用），严重限制了该方法的实际应用性。

(C3) 缺乏硬件反馈。人类专家通常通过测试和优化的迭代工作流程来开发 CUDA 内核。他们依赖于硬件反馈，例如 Nsight Compute (NCU) ² 识别性能瓶颈并相应地优化内核 [?]。相比之下，基于强化学习的方法 [?] 训练大模型以直接生成或优化内核，但完全不利用硬件反馈。因此，它们在生成过程中依赖盲目的探索，缺乏目标导向的指导。这通常导致内核效率不高，限制了其实际应用性。

这些挑战引出一个自然的问题：我们能否设计一种简单但有效的硬件感知方法，以低成本可靠地生成高效的 CUDA 内核？

我们的贡献。为应对这些挑战，我们提出 CudaForge，一种简单、高效且低成本的多智能体 workflow，用于 CUDA 内核生成与最优化，如图 2 所示。我们的工作流受到人类专家迭代工作流的启发 [?]，包含开发初始内核、测试正确性、分析硬件反馈以及迭代改进等步骤。

该 workflow 包含两个专门的大型语言模型智能体，它们迭代地生成并优化 CUDA 内核：一个编码器 (Coder)，根据任务指令和裁判 (Judge) 的反馈生成内核；一个裁判 (Judge)，分析内核与硬件反馈，以指导编码器的生成。CudaForge 的一个关键创新在于其集成了外部硬件反馈，包括 GPU 规格和 Nsight Compute (NCU) 指标，使裁判能够像人类专家一样识别性能瓶颈，并向编码器提供针对性的最优化指导。

与使用同一 LLM 生成和评估代码的单 LLM 方法相比，我们的框架将这两个角色分离为一个独立的编码器和评判器，从而实现更专业的推理和更可靠的迭代优化。不同于基于强化学习的方法，CudaForge 无需训练，避免了策略训练带来的高昂开销。它还具备硬件感知能力，能够根据潜在系统特性定制 CUDA 内核优化，使所提出的框架在不同 GPU 上具有良好的可泛化性。最后，与现

¹发表于 2025 年 9 月 16 日的 arXiv

²Nsight Compute (NCU) 是 NVIDIA 官方的 CUDA 程序内核级性能分析工具。

有的多智能体框架 [?] 相比, CudaForge 轻量且成本效益高, 在单个 RTX 6000 GPU 上仅需 26.5 分钟即可运行, 平均每个内核的 API 开销约为 0.3 美元, 同时仍实现了显著更优的性能。

我们对 CudaForge 在 Level 1 到 Level 3 的 250 个 KernelBench 任务上进行了评估。尽管这些任务具有挑战性, CudaForge 仍达到了 97.6% 的正确率, 并在平均速度上相比 PyTorch 基准提升了 $1.68\times$, 显著优于先进的强化学习模型 Kevin-32B、先进的前沿模型 OpenAI-o3 [?] 以及 Agentic Baseline [?], 如图 1 所示。此外, 我们还对 CudaForge 的特征进行了全面的消融研究, 包括其在多种 GPU 架构上的有效性、通过增加生成数量实现的推理时可扩展性, 以及不同基础模型的影响。总体而言, 我们观察到所提出的 CudaForge 在所有这些情景下均表现出稳健的性能。

这些发现突显了本工作的关键贡献: 所提出的大型语言模型智能体工作流 CudaForge 简单而高效: 以极低的成本, 为多种实际任务、各类 GPU 架构和基础模型开发出高性能的 CUDA 内核。它还展现出强大的测试时缩放能力, 随着迭代次数的增加, 解决方案质量可显著提升。这些结果证明了 CudaForge 强大的实际应用价值。

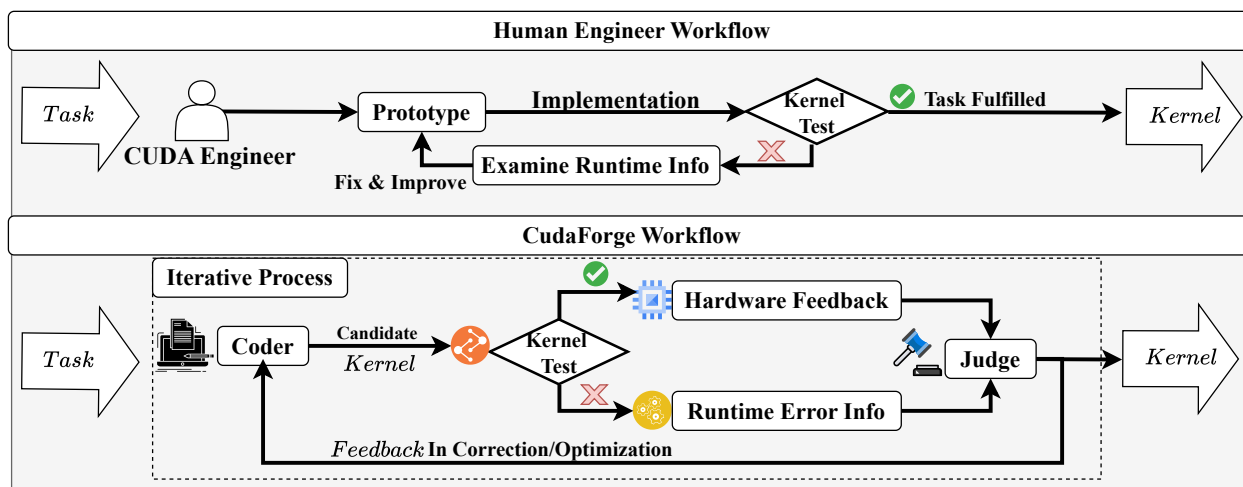


Figure 2: 人类与 CudaForge 工作流的对比。上图: 人类专家通过编写原型、测试并分析运行时反馈, 迭代优化内核。下图: CudaForge 模拟人类工作流, 采用两个专用智能体 (代码生成器和评判器)。代码生成器生成候选内核, 评判器则分析运行时信息和硬件反馈, 提供修正或最优化建议。该过程持续迭代, 直至达到最大轮次 N 。

2 CudaForge 框架用于 CUDA 核函数最优化

2.1 CudaForge 框架

给定一个 CUDA 内核生成任务, 目标是生成一个在功能上等价于其 PyTorch 参考实现的同时, 执行延迟最低的内核。

受人类专家迭代工作流的启发 [???], 我们设计了 CudaForge 作为一种迭代式多智能体框架, 如图 2 所示。该框架包含两个独立的智能体: 一个 **Coder** 和一个 **Judge**。Coder 根据任务描述和 Judge 的反馈生成候选内核, 而 Judge 则利用内核本身、硬件反馈和运行时信息对每个候选内核进行评估。

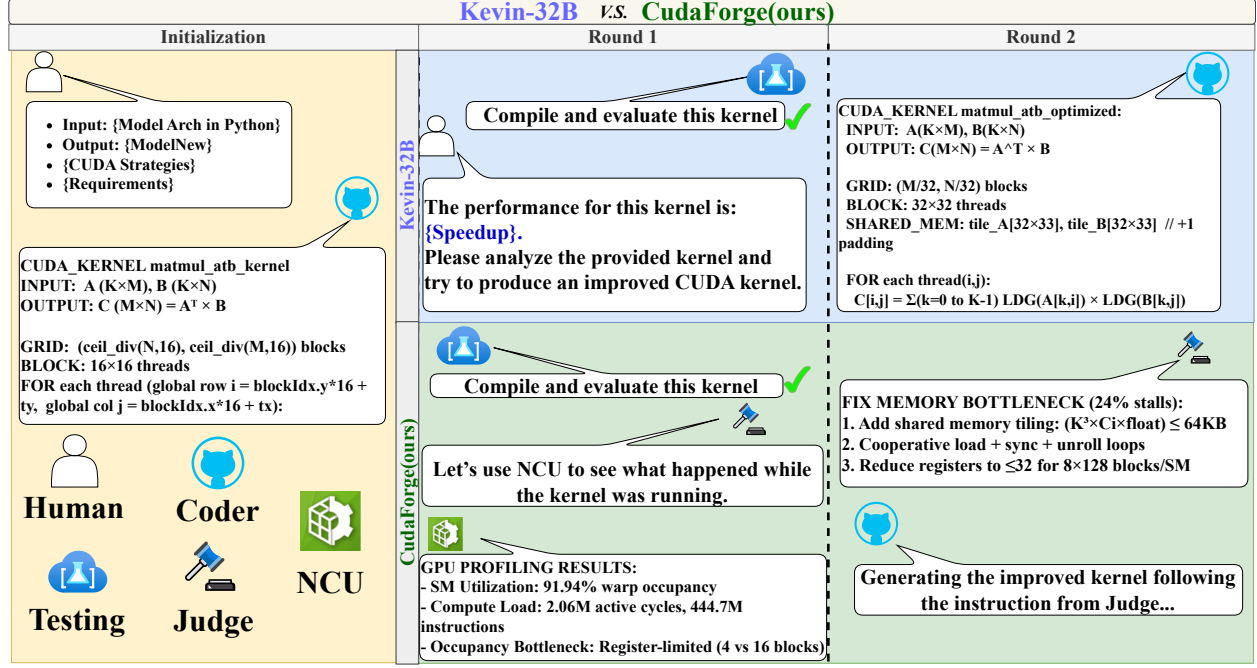


Figure 3: CudaForge 在内核最优化方面的整体流程概述，与 Kevin-32B 进行对比。上方：基于强化学习的 Kevin-32B 流水线，仅依赖文本优化，因此执行盲目的探索。下方：我们的 CudaForge 工作流，利用硬件反馈来指导内核优化。当 CudaForge 中的编码器在第一轮生成正确的候选内核时，系统使用 Nsight Compute (NCU) 对其进行性能分析，获取 NCU 指标。在第二轮中，评判器根据这些指标和 GPU 规格分析性能瓶颈（例如寄存器或内存受限），并提供有针对性的优化反馈。随后，编码器据此对内核进行改进。与仅基于加速得分进行优化的 Kevin-32B 相比，我们的框架通过硬件感知的迭代，实现了更可解释且更有效的性能提升。

更具体地，给定一个 CUDA 内核生成任务，Coder 首先接收任务需求和 PyTorch 参考实现，然后生成初始候选内核。该内核在测试用例上进行编译并执行，以检查其正确性。如果失败，Judge 会检查运行时信息（例如编译错误、与 PyTorch 参考实现输出不匹配），并分析出错的内核。随后，它返回修正反馈（例如缺少头文件），以指导下一轮改进。一旦某个内核候选通过了正确性测试，Judge 便使用 NCU 工具对其进行性能剖析，以获取 NCU 指标（例如内存吞吐量、占用率、线程束效率）。结合 GPU 规格，这些指标构成硬件反馈，使 Judge 能够识别主要瓶颈（例如计算密集型或内存密集型），并向 Coder 提供一条具体的优化反馈（例如使用共享内存）。

在下一轮中，编码器会收到之前的内核、裁判的反馈以及原始任务要求，并生成一个修正或优化后的内核。此过程最多重复 N 轮，之后我们选择最高效且正确的内核作为最终解决方案。

CudaForge 通过三个关键设计选择实现了可靠性和高效性。首先，它采用双智能体系统，其中生成智能体（Coder）专注于代码生成，评估智能体（Judge）专注于评估，将“认知”负担进行分离（参见第 3.6 节的消融实验）。生成智能体仅接收来自评估智能体的反馈，而评估智能体则利用硬件和运行时信息来指导生成与优化。这种分工模式模仿了人类工作流，有效降低了遗漏错误或低效问题的风险。其次，该框架遵循迭代优化过程，在多轮中逐步修正错误并提升效率。这一机制使得在复杂任务上也能实现稳定改进。第三，它显式地引入硬件反馈，例如 GPU 规格和 NCU 指标，使评估智能体能够精准定位性能瓶颈，并向生成智能体提供可操作的优化建议。这种有向优化避免了盲目探索，确保了性能的定向提升。

2.2 组件设计

编码器的设计 Coder 在我们的框架中充当生成模块，根据任务描述、先前的候选代码以及 Judge 模块的反馈，负责生成功能正确且高效的 CUDA 内核候选。直接让大模型从零开始生成一个 CUDA 内核是一项极具挑战性的任务 [?]。遵循 KernelBench [?] 的情景设定，我们在提示中为 Coder 提供了一个 one-shot 示范，其中包含一个 PyTorch 参考实现及其对应的 CUDA 内核实现示例。该示例作为结构和风格上的指导，帮助 Coder 理解生成有效内核所需的预期语法、内存访问模式以及 API 使用方式。

此外，我们发现管理编码器的记忆范围对于稳定且高效的生成至关重要。使用完整的对话历史来提示模型会引入过多的上下文冗余，常常导致生成错误的内核代码并增加 API 成本。为缓解这些问题，我们采用轻量级记忆设计，即编码器在每轮中仅接收最新反馈、之前的内核候选代码和任务描述，而不保留完整的对话历史。在每轮中，它仅基于最新的反馈、之前的内核候选代码和任务描述进行优化或修正，从而专注于改进前一版本。这种设计显著提升了代码的稳定性，降低了推理成本，并使模型能够执行精确的、面向硬件的最优化。详见附录 A 中编码器的提示设计。

正确性测试的设计。 在代码生成器生成候选核函数后，我们的工作流进入验证其正确性的两阶段测试过程，包括编译和执行。编译阶段确保生成的核函数在语法上是有效的，并能成功编译为可执行的 CUDA 代码。执行阶段则通过在预定义的测试用例上运行已编译的核函数，评估其功能正确性。我们将在相同输入下，将核函数的输出与 PyTorch 参考实现的输出进行比较，只有当两者输出之间的数值差异在较小容差范围内（我们在实验中设定为 $1e-4$ ）时，才认为该核函数功能正确。只有当核函数同时通过编译和执行阶段，才被视为正确，这一准则被广泛采用 [???]。

裁判员的设计。 Judge 在我们的框架中充当评估与指导模块。其主要职责是根据任务描述、当前候选核函数以及硬件反馈或运行时信息，为编码器提供可执行的反馈以指导其下一轮动作。Judge 根据核函数的有效性在两种不同模式下运行：若核函数编译失败或产生错误输出，Judge 进入修正模式，识别诸如无效索引、竞争条件或缺失头文件等问题，并返回 **修正反馈**，指导编码器进行修复。若核函数通过所有正确性检查，Judge 则进入优化模式，利用硬件反馈识别主要性能瓶颈——例如内存受限、计算受限或占用率受限——并据此制定 **优化反馈**。有关如何使用硬件反馈的详细信息，请参见第 2.3 节。

与代码生成器 (Coder) 的设计类似，评判器 (Judge) 不会保留完整的对话历史。在每一轮中，评判器根据相关模式被提示，并按照相应的角色进行操作。最终，它生成结构化的反馈信息，以 JSON 格式输出，随后被提取并传递给代码生成器，用于下一轮的处理。这种设计确保了智能体之间交互的模块化和清晰性，使评判器能够高效地将硬件级别的性能分析信号转化为精确且可解释的反馈，从而驱动内核的迭代优化。详见附录 A 中评判器的提示内容。

2.3 如何集成硬件反馈

在本小节中，我们详细描述了一个关键的设计考量，该考量使得 CudaForge 能够利用硬件反馈来优化内核性能。硬件反馈 将静态的 GPU 特性（例如架构、内存带宽、每线程寄存器限制、每 SM 共享内存容量）与 Nsight Compute (NCU) 在内核执行过程中收集的性能指标（例如内存吞吐量、占用率和线程束效率）相结合。通过交叉参考 GPU 特性与 NCU 指标，Judge 推断出内核的主要性能瓶颈及其根本原因。图 3 展示了 Judge 如何利用硬件反馈来优化内核。

正如 CUDA 工程师关注关键指标一样，我们选择不将 NCU 的所有指标传递给评论员。输入全部指标可能会使决策过程因过多且部分冗余的信号而不堪重负，导致判断不稳定（详见第 3.6 节和附录 B.1 中的消融研究与案例分析）。相反，我们设计了一种新的协议，对 NCU 提供的关键指标子集进行采样，并将其转发给评论员，从而提升评论员输出的质量。更具体地，关键指标子集是在离线阶段（工作流启动前）通过以下步骤选取的：

(Step 1) 内核采样与选择：我们首先在一些预先选定的代表性任务（例如，Conv2D、MatMul）上分析关键指标，以准备一个可靠的指标集合。具体而言，对于每个任务，我们使用单个 SOTA 模型（例如，OpenAI-o3）运行 100 次自精炼（重复执行生成 → 执行/采样 → 评估 → 修复/优化）过程，收集生成的和正确的内核，并选择 10 中速度差异最大（最快与最慢）的内核。详见算法 1。

Algorithm 1: Step 1: Kernel Sampling and Selection

Input: Task set $Task = \{T_1, T_2, \dots, T_n\}$ **Output:** Selected subsets K_i^* for each task T_i

```
for  $i \leftarrow 1$  to  $n$  do
     $K_i \leftarrow \emptyset$ ;
    for  $j \leftarrow 1$  to 100 do
         $k_j \leftarrow \text{generate\_kernel}(T_i)$ ;
         $K_i \leftarrow K_i \cup \{k_j\}$ ;
    end
    Sort  $K_i$  in nondecreasing order according to kernel runtime;
     $m \leftarrow |K_i|$ ; // Here  $m = 100$ 
     $K_i^* \leftarrow \{K_i[1], K_i[2], K_i[3], K_i[4], K_i[5], K_i[m-4], K_i[m-3], K_i[m-2], K_i[m-1], K_i[m]\}$ ;
end
```

(Step 2) 每个任务中的前 20 个指标: 随后, 我们对每个任务内的指标进行精炼, 以识别最相关的候选指标。具体而言, 对于每个任务, 我们将步骤 1 中选定的 10 个内核的 NCU 指标合并成一个单一数据集。由于 Nsight Compute 在所有内核上报告一致的完整指标集合, 因此指标类别默认是对齐的。然后我们移除别名和强共线性指标, 并计算每个指标与内核运行时间之间的皮尔逊相关系数。我们仅保留相关系数绝对值最高的前 20 个指标作为该任务的候选指标集 (详见附录 B.2 中的示例)。

(步骤 3) 任务间指标选择: 最后, 我们整合各任务的指标, 构建一个稳定且与任务无关的指标集。我们对比各任务的前 20 个指标列表, 保留那些持续出现、相关系数方向一致且全局得分较高的指标。具体而言, 对于每个指标, 我们计算其全局相关系数得分, 定义为该指标在所有任务中与运行时间的绝对皮尔逊相关系数的平均值。随后, 我们选取全局得分超过所有候选指标第 75 百分位数 (P_{75}) 的指标, 确保仅保留相关性最强的指标。这最终得到 24 个在各任务中均与核函数运行时间强相关的指标。之后, 判别器将使用 NCU 对每个生成的核函数进行剖析, 并仅以这 24 个指标作为参考 (详见附录 B.3 中的完整指标列表)。详见算法 2。

Algorithm 2: Step 2-3: Profiling and Metrics Selection

Input: $K^* = \{K_1^*, K_2^*, \dots, K_n^*\}$, where each $K_i^* = \{k_1^*, k_2^*, \dots, k_{10}^*\}$
Output: Final metrics set $Final_Metrics$

$M^* \leftarrow \emptyset$;
for $i \leftarrow 1$ **to** n **do**
 $M_i^* \leftarrow \emptyset$;
 foreach $k \in K_i^*$ **do**
 $M \leftarrow \text{NCU_Profile}(k)$; // Run NCU profiling, $M = \{m_1, m_2, \dots, m_j\}$
 foreach $m \in M$ **do**
 $r_{m,i} = \text{Compute Pearson correlation coefficient } r(m, \text{runtime}(k))$;
 end
 $Top20(k) \leftarrow \text{the 20 metrics in } M \text{ with highest } |r(\cdot, \text{runtime}(k))|$;
 $M_i^* \leftarrow M_i^* \cup Top20(k)$;
 end
 $M^* \leftarrow M^* \cup M_i^*$;
end
// Compute global correlation scores across tasks
foreach $metric\ m \in M^*$ **do**
 Compute $S_m = \frac{1}{n} \sum_{i=1}^n |r_{m,i}|$; // $r_{m,i}$: Pearson correlation between metric m and
 runtime on task i
end
// Select stable and highly correlated metrics
 $Final_Metrics \leftarrow \{m \mid m \text{ appears in multiple tasks, keeps same sign, and } S_m > P_{75}(S)\}$;
// Final set contains 24 distinct metrics

在离线确定了 NCU 指标的关键子集后，评论员将使用这些指标来识别 CudaForge 工作流程中的性能瓶颈。在每次最优化迭代中，评论员使用 NCU 对生成的核函数进行剖析，并收集硬件反馈信息，包括静态 GPU 规格以及 NCU 指标的关键子集。基于这些信息，评论员识别出当前核函数中的主要瓶颈。

为防止智能体在无方向的情况下进行推理并生成次优结果，评论员被提示在每轮仅根据自身推理捕捉 3-4 个最重要的指标。例如，当发现内存吞吐量较高但计算资源利用率较低时，评论员可判定当前核函数为内存受限型，进而选择与内存相关的指标作为本轮的关键指标。随后，评论员将生成关于如何修改核函数以解决当前关键瓶颈的建议。编码器将在下一轮生成中相应地采纳这一指导。该机制使我们的多智能体系统能够在每轮中仅聚焦于解决一个关键程序瓶颈，从而通过迭代轮次逐步优化整体核函数性能，如同人类专家的真实工作流一般。

3 实验

3.1 基准和评估

我们在 **KernelBench** [?] 上评估了我们的方法和基准方法，这是一个用于评估大模型生成 CUDA 内核能力的流行基准。KernelBench 包含多个难度等级，我们采用了第 1 级到第 3 级的所有任务，共计 250 个任务。具体而言，第 1 级包含相对简单的 100 个任务，涉及基本操作（例如矩阵乘法），第 2 级包含中等难度的 100 个任务，由多步操作组合构成，第 3 级包含 50 个具有挑战性的任务，涉及完整的神经网络架构（例如 AlexNet）。每个任务都配有参考的 PyTorch 实现和预定义的输入/输出规范，从而能够实现完全自动化且可靠的正确性与性能评估。KernelBench 的详细信息见附录 D.1。

由于某些消融研究运行实验的计算成本较高，我们并未在完整的 KernelBench 基准上对其进行评估。相反，我们通过从 KernelBench 的每个难度等级中成比例采样，构建了一个分层随机子集的任务集合，记为 $\mathcal{D}^*$ 。这确保了 $\mathcal{D}^*$ 保持与完整基准相同的任务分布，同时实现了公平且高效的评估。具体而言， $\mathcal{D}^*$ 共包含 25 个任务，其中 Level 1 有 10 个任务，Level 2 有 10 个任务，Level 3 有 5 个任务。关于 $\mathcal{D}^*$ 构建的更多细节见附录 D.2。

我们使用以下指标在 KernelBench 上评估模型性能：(1) **正确性**：生成的内核能够成功编译并在所有测试用例上产生与 PyTorch 参考输出相同的任务比例。(2) **性能**：在特定 GPU 上测试的执行速度比值，即正确生成的内核与其 PyTorch 参考版本之间的执行速度比。(3) **快速₁**：执行速度超过其 PyTorch 参考版本的正确内核的比例。(4) **中位加速比**：所有任务中“性能”值的中位数，反映典型而非平均行为。(5) **第 75 百分位加速比**：性能值的第 75 百分位数，捕捉上四分之一效率。

对于执行迭代优化或生成多个候选解的方法（包括 CudaForge），我们报告每个任务中所有候选解里表现最佳的正确内核。

3.2 情景 & 基准

在我们的主要结果中，我们将 CudaForge 实例化为使用 OpenAI-o3 作为编码器和评判者，作为我们的默认情景。我们将最大迭代轮数设置为 $N=10$ ，以平衡性能提升与推理成本。除非另有说明，所有方法均在 Quadro RTX 6000 的相同编译/运行环境和特定任务测试套件下进行评估。

为了对 CudaForge 的性能进行上下文分析并评估先进基础模型的影响，我们在主结果和消融研究中包含了以下基准：

- OpenAI-o3：使用 OpenAI-o3 进行一次生成，无需迭代；
- o3-self-refine（我们的基准）：使用 OpenAI-o3 进行十轮自精炼，不使用裁判模型，模型仅依靠自身根据硬件反馈来修正和优化内核；
- o3-校正（我们的基准）：CudaForge 的一个变体，其中 Judge 仅提供正确性反馈，不提供最优优化反馈；

Table 1: KernelBench 主要结果（第 1-3 级，共 250 个任务）。Agentic 基准的结果位于第 1 和第 2 级。此处所有实验均在 RTX 6000 上运行。在 $\mathcal{D}^*$ 上评估的方法以 * 标记。CudaForge-Scaling Up 表示增加最大迭代轮次

Method	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast $_1\uparrow$
OpenAI-o3	57.6%	0.390	1.014	0.680	31.60%
o3-self-refine	90.8%	1.012	1.209	1.107	55.20%
o3-correction	97.6%	1.031	1.238	1.222	59.60%
o3-optimization	88.4%	1.061	1.483	1.509	64.00%
Agentic Baseline(Level 1 & 2)	95.0%	—	—	1.490	—
CudaForge(full metrics)*	100%	1.280	1.489	1.414	80.00%
CudaForge	97.6%	1.107	1.592	1.677	70.80%
CudaForge(Level 1 & 2)	98.0%	1.112	1.617	1.776	71.50%
CudaForge*	100%	1.322	1.736	1.767	84.00%
CudaForge-Scaling Up*	100%	1.317	1.777	2.265	92.00%

Table 2: CudaForge 在 RTX 6000 上关于 KernelBench（Level 1-3，250 个任务）的主要结果。

Task	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast $_1\uparrow$
Level 1	96%	1.044	1.751	1.448	54.0%
Level 2	100%	1.124	1.427	2.104	89.0%
Level 3	96%	1.081	1.510	1.283	68.0%

- o3-最优化（我们的基准）：CudaForge 的一个变体，其中 Judge 仅提供最优化反馈，不提供修正反馈；
- Kevin-32B：一个基于强化学习的强模型，用于从 [?] 进行 CUDA 内核最优化。我们直接采用其官方论文中的结果。
- Agentic 基准：来自 [?] 的 Agentic 工作流，这是一种强大的多智能体基准，使用包括推理型（o3 & o4-mini）和传统大模型（Claude Sonnet 3.7 & GPT-4.1）的 LLM 集成。
- CudaForge (完整指标)：CudaForge 的一种变体，其中判断器利用全部 NPU 指标。

此基准设置使得以下方面的全面比较成为可能：(i) 基础模型与对应的智能体方法，(ii) 判定反馈的有无，(iii) 基于强化学习的方法与无需训练的智能体方法，以及 (iv) 不同的 Agentic 方法。

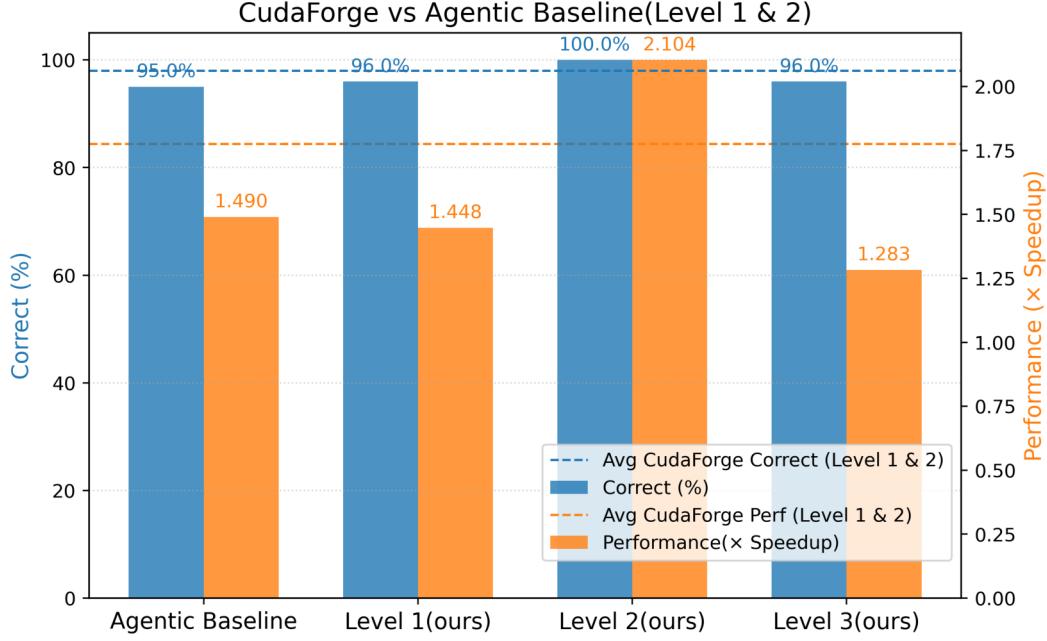


Figure 4: 在 KernelBench 上, CudaForge 与 Agentic 基准的正确性与性能对比。虚线表示 CudaForge 在 Level 1 和 Level 2 上的平均结果。CudaForge 在 KernelBench Level 1 和 Level 2 上均优于 Agentic 基准, 并且在 Level 3 上也取得了优异的表现。

3.3 RTX 6000 的主要成果

表 1 报告了 KernelBench 中的主要结果。CudaForge 在整个 KernelBench 以及分层子集 $\mathcal{D}^*$ 中, 各项指标上均持续优于所有基准方法。

在 KernelBench 上, CudaForge 的正确率达到了 **97.6%**, 平均性能为 **1.677 $\times$** , 且 **70.8% Fast₁**, 同时实现了中位数加速比 1.107 $\times$, 第 75 百分位加速比为 1.592 $\times$ 。这些结果相较于基础模型 OpenAI-o3 以及其他消融变体 (包括 o3-self-refine、o3-correction 和 o3-optimization) 均有显著提升。

在缩减数据集 $\mathcal{D}^*$ 上, CudaForge 达到 100% 的正确性, 中位数加速比为 1.322 $\times$, 75% 分位数加速比为 1.736 $\times$, 平均性能为 1.767 $\times$, 以及 84.0% Fast₁。这大幅超过智能体 (完整指标), 后者仅达到 1.280 $\times$ 中位数, 1.489 $\times$ 75% 分位数, 1.414 $\times$ 平均性能, 以及 80% Fast₁。此外, 在缩放最大迭代轮次后, CudaForge-缩放实现了更强的性能, 加速比为 **2.265 $\times$** 。我们将在第 3.6 节进一步讨论使用 $\mathcal{D}^*$ 的消融实验, 以及在第 3.7 节讨论缩放最大迭代轮次。

我们还对比了 CudaForge 与 Agentic 基准³ 在 KernelBench Level 1 和 Level 2 上。如表 1、2 和图 4 所示, CudaForge 实现了 98% 的正确率和平均 1.776 $\times$ 的加速比, 优于 Agentic 基准 (95.0%, 1.490 $\times$), 尤其是在加速比方面表现更优。该结果表明我们在与现有 Agentic 工作的对比中具有明显优势。

³请注意, 它仅报告一级和二级的结果, 由于论文尚未开源代码, 我们直接采用论文中的结果。

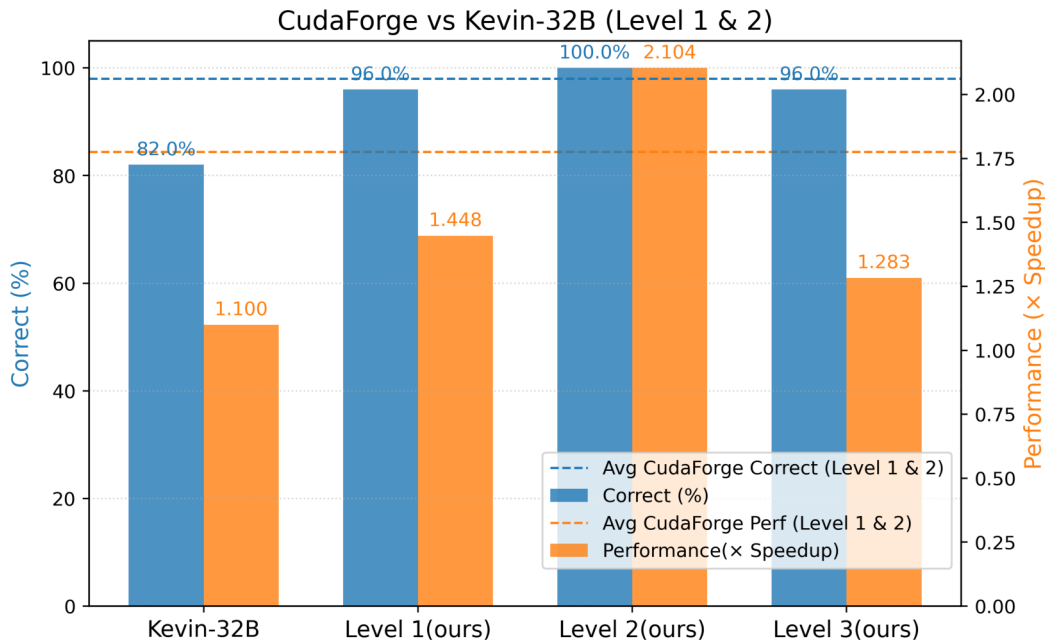


Figure 5: 在 KernelBench 上，CudaForge 与 Kevin-32B 的正确性与性能对比。虚线表示 CudaForge 在 Level 1 和 Level 2 上的平均结果。尽管无需训练，CudaForge 在 KernelBench Level 1-2 中的表现仍优于 Kevin-32B，且在 Level 3 中取得了卓越的结果。

值得注意的是，在代表 KernelBench 中最具挑战性级别的第 3 级，CudaForge 实现了 **96%** 的正确率和平均 **1.283×** 的加速比。鉴于第 3 级任务的复杂性，这些任务涉及完整的神经网络结构和多阶段操作，上述结果表明，CudaForge 能够可靠地生成和优化高度复杂的 CUDA 内核，而此前的方法 [??] 未曾探索过这一领域。

3.4 与 Kevin-32B 在 H200 上的对比

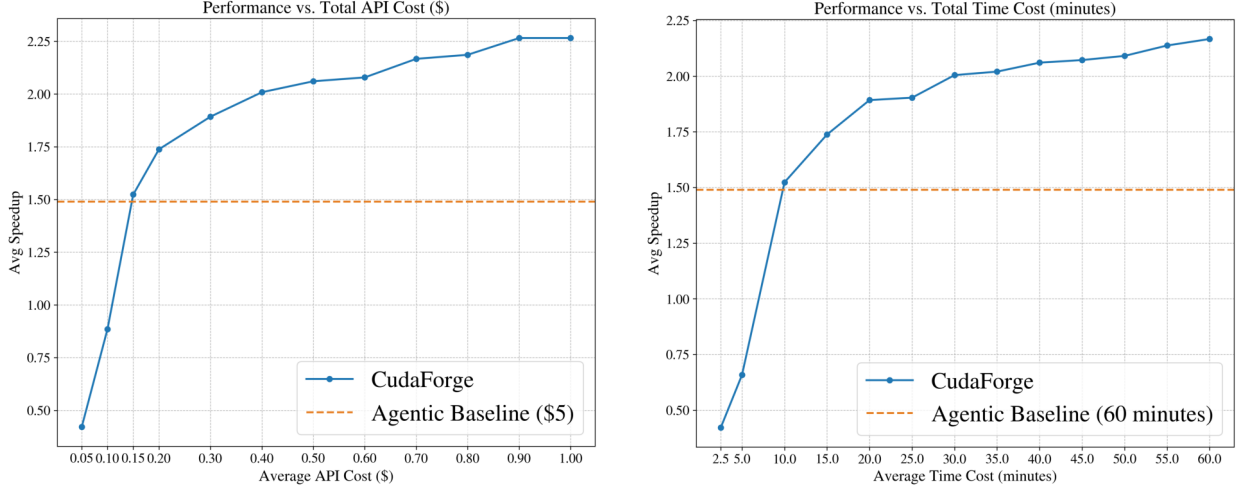
在本小节中，我们对比了 CudaForge 与 Kevin-32B [?]。然而，由于 Kevin-32B 并未开源——这意味着测试时的提示、评估设置和基准规格等关键细节并未公开——因此无法进行完全公平且可复现的对比。根据原始论文，Kevin-32B 在 H200 GPU 上使用一个自构建的基准进行训练和评估，该基准的难度与 KernelBench Level 1 和 Level 2 相当。为在这些限制条件下尽可能确保对比的公平性，我们在相同的 H200 硬件上对 CudaForge 重新进行了评估，覆盖 KernelBench Level 1–3，尽可能使我们的计算环境和评估协议与 Kevin-32B 报告的情况保持一致。

如图 5 所示，CudaForge 在所有级别上均表现出一致更高的正确率和性能。在与 Kevin 的基准任务相当的级别（1 级和 2 级），CudaForge 平均达到 **98.0%** 的正确率和 **1.662×** 的性能，优于 Kevin-32B 的 82.0% 正确率和 1.10× 速度提升。即使在最具挑战性的第 3 级任务中，CudaForge 仍保持优异表现，正确率为 **96.0%**，性能达 **1.261×**，展现出其在复杂 CUDA 内核上的鲁棒性。

这些发现表明，尽管是一个无需训练的框架，CudaForge 在可靠性和效率方面均超过了经过强化学习训练的 Kevin-32B。我们将其优势归因于对硬件反馈的有效利用以及工作流的设计，这使得

Table 3: CudaForge 与 Agentic 基准 [?] 之间 API 调用和计算时间成本的对比。

Method	Metric	Average	Level 1	Level 2	Level 3
Agentic Baseline	API Cost (\$)	5.0	—	—	—
	Time (min)	60.0	—	—	—
CudaForge	API Cost (\$)	0.30	0.29	0.30	0.33
	Time (min)	26.5	28.5	24.1	27.1



(a) CudaForge 的性能与 API 开销对比。

(b) CudaForge 的性能与计算时间对比。

Figure 6: CudaForge 的成本与性能之间的关系。API 调用次数和计算时间均与性能呈单调相关。CudaForge 在每项任务使用不超过 \$0.15 和 10 分钟 的情况下，已超越 Agentic 基准。

CudaForge 能够执行有针对性的硬件感知最优化，而非仅仅依赖于（可能低效的）强化学习训练。

3.5 CudaForge 的 API 与计算时间开销

我们评估了 CudaForge 在 KernelBench 数据集 $\mathcal{D}^*$ 上的 API 成本和计算时间成本。API 成本被定义为每个内核生成任务的总支出，而计算时间 则被定义为端到端的墙上时钟时间，包括内核编译和执行、模型生成以及 NCU 性能分析。所有实验均在单个 RTX 6000 GPU 上进行。

如表 3 所示，CudaForge 在单个 RTX 6000 GPU 上平均仅需 26.5 分钟 的时钟时间，并且每个内核的 API 成本仅为 \$0.3。与 Agentic 基准 [?] 相比，其成本效率极高，后者在附录 E 中报告每个内核需要约 6 个 GPU 小时 的 H100 计算时间和 \$5 的成本。

我们进一步分析了 API 开销和计算代价如何影响 CudaForge 的性能，如图 6a 和 6b 所示。随着 API 和计算时间的增加，CudaForge 的性能呈单调提升。值得注意的是，CudaForge 在每个任务使用不超过 \$0.15 和 10 分钟 的情况下，已超越 Agentic 基准 [?]，展现出强大的成本-性能权衡能力。

我们把 CudaForge 的成本效益归因于三个主要因子：

1. **利用硬件反馈。** Judge 利用硬件反馈来诊断当前的性能瓶颈，并向 Coder 提供有针对性的、可执行的最优化指导。这种聚焦的优化避免了盲目探索，加速了收敛，从而减少了 API 调用次数和性能分析轮次。
2. **选择性 NCU 指标。** 与分析全部指标集不同，CudaForge 采用了一组经过筛选的关键 NCU 指标。这不仅缩短了 NCU 分析时间，还降低了 API 成本，因为较少的指标减少了 Judge 查询中的输入 token 长度。
3. **轻量级记忆。** 在每次迭代中，编码器和评判器均仅接收当前轮次的信息，而非使用完整的话历史。这种轻量级记忆设计减少了冗余的上下文 token 和计算开销，使每个智能体能够专注于最近的反馈信息和候选核函数。

3.6 消融研究

与使用全部 NCU 指标的比较 CudaForge 的一个关键设计选择是过滤完整的 NCU 指标集合，仅保留 24 个关键指标供评论员使用。这种有选择性的设计使评论员能够专注于最具信息量的性能指标，避免冗余，从而实现更一致的最优化反馈。我们进行了消融实验以评估这一设计选择。如表 1 所示，使用完整的 NCU 指标集合会导致正确率和性能下降，因为评论员被过多且部分冗余的信号所淹没。此外，使用所有 NCU 指标进行分析会显著增加推理开销——每个核函数在 RTX 6000 GPU 上大约需要 40 分钟，并产生约 1 美元的 API 成本；而我们的选择性指标设计则以更低的开销（在 RTX 6000 上仅需 26.5 分钟，API 成本约为 0.3 美元）实现了更优的性能。这些结果证实，简洁且聚焦的硬件反馈不仅更有效，也更高效。我们还在附录 B.1 中提供了案例研究。

与 o3-self-refine 的比较。 CudaForge 背后的一个关键动机是将生成与评估的角色解耦。在 o3-self-refine 中，同一个模型执行了十轮自精炼，隐式地承担了双重角色：它既要提出新的内核，又要根据硬件反馈和运行时信号评估自身输出。尽管这种策略将正确率从 57.6% 提升至 92.8%，但性能仍受限 ($1.107\times$ 加速比, 55.2% 快速₁)。相比之下，CudaForge 明确分离了职责：编码器专注于代码生成，而评论员则专精于提供结构化反馈。这种分工至关重要——使每个智能体能够专注于不同的推理过程——从而显著提升了效率 ($1.677\times$ 加速比, 70.8% 快速₁)，且未牺牲正确性。

与 o3 校正（仅校正判断器）的比较。 在 o3-correction 情景中，Judge 仅根据运行时信号提供修正反馈，而不提供最优化反馈。该设置实现了与 CudaForge 相同的 97.6% 正确率，证实了迭代错误修正足以确保可靠的核函数生成。然而，效率仍显著较低，仅达到 $1.222\times$ 性能和 58.8% Fast₁。与 CudaForge($1.677\times$, 70.8%) 的对比表明，尽管正确性反馈能稳定生成过程，但基于硬件性能分析的效率反馈对于实现显著的效率提升至关重要。

与 o3-最优化（仅最优化判断）的比较。 我们还评估了裁判仅提供最优化反馈而无修正反馈的变体情景。在此情景下，编码器经常生成无法编译或运行的内核，因为功能错误未被纠正。因此，整体正确性显著低于 CudaForge，且最优化指导的潜在优势无法实现。这一结果表明，正确性反馈是

Table 4: CudaForge 在不同 GPU 上的性能表现。该系统通过在最优化过程中融合 GPU 规格和 *Nsight Compute* 性能分析信号，始终在各类架构上实现高正确性与强劲性能。

GPU	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast ₁ $\uparrow$
RTX 6000(Ada Arch-Data center level)	100%	1.322	1.736	1.767	84.0%
RTX 4090(Ada Arch-Desktop level)	100%	1.188	1.589	1.327	80.0%
A100(Ampere Arch-Data center level)	100%	1.371	1.762	1.841	84.0%
RTX 3090(Ampere Arch-Desktop level)	100%	1.155	1.706	1.320	72.0%

前提条件：在未确保功能有效性之前，仅凭最优化反馈无效且常常浪费。相比之下，CudaForge 同时利用修正反馈和最优化反馈，实现了稳定的内核生成和持续的效率提升。

3.7 CudaForge 的泛化能力

在本节中，我们分析了 CudaForge 在不同最大迭代次数 N 、GPU 架构和基础模型下的能力。考虑到完整实验的高成本，本节使用分层子集 $\mathcal{D}^*$ 。

增大最大迭代轮数。我们研究了最大迭代次数 N 对 CudaForge 性能的影响。

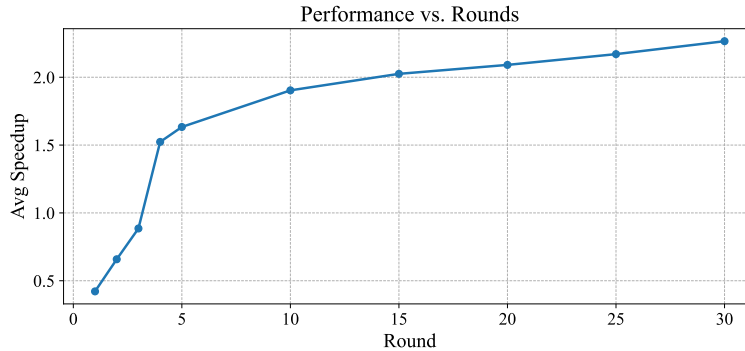


Figure 7: 将迭代轮数缩放至 30，在 KernelBench（子集 $\mathcal{D}^*$ ）上。

如图 7 所示，将 N 从 1 增加到 10 可带来显著的性能提升，表明 CudaForge 可通过迭代优化快速提高内核效率。进一步将 N 从 10 增加到 30 仍能持续改善性能，但增长速率逐渐放缓，说明系统正逐步接近其性能上限。经过 30 轮最优化后，CudaForge 将平均加速比提升至 $2.271\times$ 。这些结果表明，CudaForge 在缩放时具有优势，并且在更大的 N 以及额外推理开销的前提下，仍有潜力实现更强的性能表现。

在不同 GPU 上使用 CudaForge 我们还在多种 GPU 架构上评估了 CudaForge，包括 RTX 6000、RTX 4090、RTX 3090 和 A100，以检验其在不同硬件条件下的有效性。如表 4 所示，CudaForge 在所有测试的 GPU 上均保持高正确率和强劲性能。这是其设计的直接结果：在最优化阶段，Judge 显式地将硬件反馈（包括 NCU 指标和 GPU 规格）纳入生成反馈给 Coder 的过程中。这使得 Coder 能够在推理时生成针对目标 GPU 定制的内核，而无需训练。

Table 5: CudaForge 在不同基础模型组合下的性能表现。我们固定一个智能体为 OpenAI-o3（记作 O3），并将另一个智能体替换为各种模型。所有组合均取得了优异结果，表明该框架并不依赖于特定的基础模型。

Models (Coder/Judge)	Correct $\uparrow$	Median $\uparrow$	75% $\uparrow$	Perf $\uparrow$	Fast ₁ $\uparrow$
O3 / O3	100%	1.322	1.736	1.767	84.0%
O3 / GPT-5	100%	1.131	1.561	2.114	96.0%
O3 / Claude-Sonnet-4	100%	1.265	1.456	1.829	84.0%
O3 / GPT-OSS-120B	100%	1.226	1.490	1.364	76.0%
GPT-5 / O3	100%	1.125	1.388	1.896	72.0%
Claude-Sonnet-4 / O3	88%	1.052	1.207	1.398	56.0%
GPT-OSS-120B / O3	96%	1.080	1.477	1.653	68.0%
QwQ / O3	84%	0.965	1.153	0.790	44.0%

使用各种大语言模型实例化 CudaForge 为了检验 CudaForge 是否依赖于特定的基模型，我们通过固定一侧（编码器或评估器）为 *OpenAI-o3*（记作 O3），并将另一侧替换为多种先进的大模型，包括 *QwQ-32B*、*GPT-5*、*Claude-Sonnet-4* 和 *GPT-OSS-120B*，开展实验。如表 5 所示，所有组合均表现出高正确率和强劲性能，与原始的 O3/O3 配置相当，甚至更优。这些结果表明，CudaForge 并不绑定于特定的基模型：其有效性源于编码器与评估器的工作流设计，随着更强模型的出现，它能够轻松受益。

4 案例研究

在本节中，我们通过一个单一任务的案例研究，说明裁判如何诊断问题并提出优化建议。图 8 展示了 CudaForge 在任务 95_CrossEntropyLoss 上的 10 轮优化过程。我们重点展示了四个具有代表性的轮次——三个优化轮次和一个修复轮次，以说明裁判如何利用来自 NCU 的硬件反馈，提供有针对性的优化或错误修复建议。

在第二轮，即最优化轮次中，裁判注意到有 23.7% 的活跃线程束因屏障型依赖而停滞，这意味着大约四分之一的潜在问题机会被同步所阻塞。根据这一情况，裁判建议将原先需要多次块级同步的共享内存归约替换为线程束级洗牌归约，并给出如下提示供编码参考：在最大值和求和阶段使用线程束级洗牌，然后执行一次跨线程束合并，将每块的 `__syncthreads()` 次数从 16 次减少到 2 次（减少了 14 次）。应用此更改后，性能从 **1.66×** 提升至 **2.42×**，屏障停滞减少，指令发射效率提高。

在第 5 轮中，这是一个修正轮次。上一轮在数值检查中失败，出现以下错误：“输出不接近，表明结果不匹配”。裁判诊断出根本原因是线程 0 中的 `target_logit` 未初始化（“Thread-0 使用了未初始化的 `target_logit`”），这意味着变量 `target_logit` 未被更新到线程 0，导致计算结果错误。因

KernelBench Level 1 Task 95: Judge Outputs & Speedup

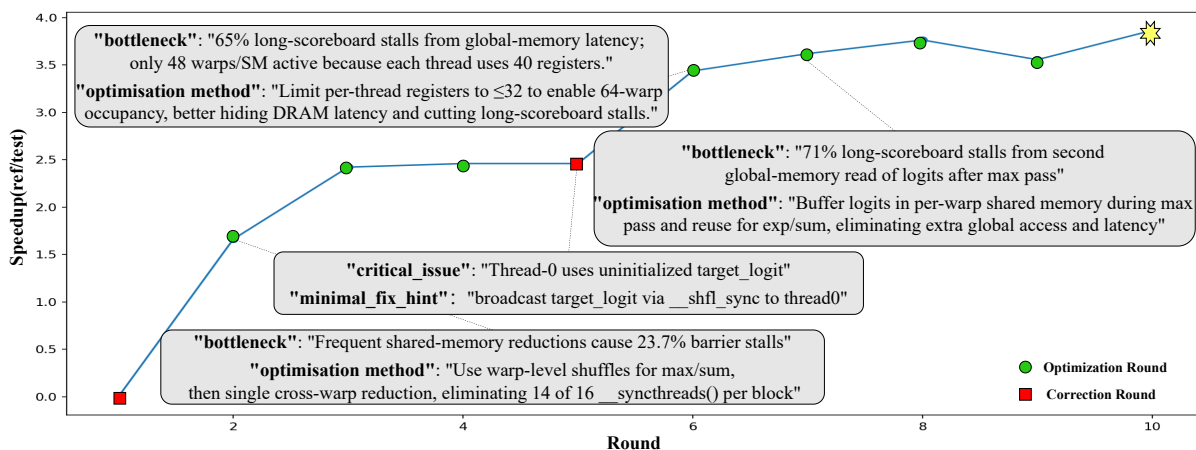


Figure 8: 裁判输出示例——在 KernelBench Level-1 任务 95 (CrossEntropyLoss) 上的瓶颈诊断与最优化建议，以及各轮次的加速效果（绿色 = 最优化，红色 = 修正）。

此，裁判给出了最小修复建议，通过 `__shfl_sync` 将 `target_logit` 广播至线程 0。应用该修复后，数值问题消失。

在第 6 轮和第 7 轮(均为最优化轮次)中, Judge 继续跟踪 `smsp_warp_issue_stalled_long_scoreboard_per_w` 指标。在第 6 轮, 该指标约为 65%, 主要反映由全局内存延迟引起的长流水线停滞。每线程寄存器使用量较高, 导致占用率受限 (仅 ~ 48 个活跃线程束/SM), 无法充分隐藏延迟。建议减少每线程的寄存器使用量, 以提升并发度至 ~ 64 个线程束/SM, 从而降低长流水线停滞占比。在第 7 轮, 该指标上升至约 71%, 根源在于最大值遍历后对 Logit 的第二次全局读取。因此, Judge 建议在最大值遍历阶段将 Logit 缓存在每线程束的共享内存中, 并在指数/求和阶段复用, 从而消除冗余的全局内存访问。综上所述, 这些策略显著减少了全局内存访问, 大幅降低了长流水线停滞, 提升了指令发射效率与吞吐量; 经过这两轮优化后, 加速比从 $3.436\times$ 提升至 $3.762\times$ 。

此任务展示了我们的 CudaForge 的稳定性与专家级工作流: 首先从硬件反馈中分析瓶颈, 然后推导出相应的最优化策略。

5 补充观测

CUDA-L1 结果中的观察。我们仔细检查了 CUDA-L1 报告的内核输出 (详见附录 C), 发现了一个有趣的现象, 我们将其称为 "虚假内核"。这些内核虽然被报告为性能优异, 但通常不包含任何实际的 CUDA 代码。相反, 它们依赖于 try-except 构造, 并回退到 PyTorch 的官方实现来完成任任务。这一观察凸显了评估大模型生成的 CUDA 内核时面临的一个根本性挑战。为避免此问题, 我们在实验中对所有内核进行了手动检查。

6 结论

我们提出了 CudaForge，一种无需训练的多智能体框架，用于 CUDA 内核生成与最优化。该框架模拟了人类专家的迭代工作流，显式地融入硬件反馈以指导针对性的内核优化，而非盲目探索。在 KernelBench 基准测试中，CudaForge 相较于所有现有方法均实现了最高的正确率和显著的性能提升，同时在多种 GPU 架构和基础大模型上表现出良好的鲁棒性。此外，其性能随着优化轮次的增加而有效提升。最后，由于其低 API 和时间开销，CudaForge 为自动化 CUDA 内核开发提供了一种实用且高效的解决方案。

A CudaForge 工作流程中使用的提示描述

A.1 第一轮生成时给程序员的提示

我们采用 *One-shot Baseline Prompt* (如 KERNELBENCH 中所介绍) 作为所有基线方法及我们方法第一轮生成的初始提示。完整提示如下所示。

Prompt for Coder in first round of generation

你编写自定义 CUDA 内核来替换给定架构中的 PyTorch 操作符以获得加速。你可以自由选择要替换的操作符集合。你可以决定将某些操作符替换为自定义 CUDA 内核，而其他操作符保持不变。你可以用自定义实现替换多个操作符，考虑操作符融合的机会（例如，将 `matmul+relu` 合并为单个内核），或采用算法上的改进（如在线 Softmax）。你唯一的限制是你的想象力。以下是一个示例，展示如何在 PyTorch 中内联嵌入自定义 CUDA 操作。

给出的示例架构是：

```
{few_base}
```

示例新架构搭配自定义 CUDA 内核如下：

```
{few_new}
```

您将获得以下架构：

```
{arch_src}
```

Model 架构使用自定义 CUDA 算子进行优化！将你的优化后输出架构命名为 ModelNew。在代码块中输出新代码。请生成真实代码，不是伪代码。确保代码可编译且完全可用。仅输出新的模型代码，不输出其他文本，也不要测试代码！

A.2 法官提示

在我们为裁判智能体设计的提示中，将角色说明和输出模式置于系统提示中。受 [???] 启发，我们设计了修正和最优化的提示，如下所示。

Prompt for CUDA Kernel Correction

你是资深的 CUDA + PyTorch 正确性审查员。你的任务是阅读一份 PyTorch 参考代码和一份 CUDA 候选代码，并报告 CUDA 代码中导致与 PyTorch 参考代码行为不一致的唯一一个最严重的正确性问题。回答应简洁且精确。

规则：

- 总回报 唯一一个问题——影响最大的单一问题。
- 优先考虑语义或正确性问题，而非微观优化或样式。

- 如果存在多个问题，请选择对输出或梯度影响最大的那个。
- 如果没有发现明显错误，请明确说明。
- 保持每个字段简洁；避免额外评论、列表或替代方案。

输出格式 (JSON):

```
{
  "critical_issue": "<max 20 words>",
  "why_it_matters": "<max 35 words>",
  "minimal_fix_hint": "<max 20 words>"
}
```

您已获得:

ERROR_LOG:

{ERROR_LOG}

PyTorch 参考 (真实值):

{PYTORCH_CODE}

CUDA 候选者 (待审计):

{CUDA_CODE}

遵循规则并以指定格式生成 JSON。

Prompt for CUDA Kernel Optimization

目标：在给定的 GPU 规格、PyTorch 参考代码、当前 CUDA 候选实现及 Nsight Compute 性能指标的基础上，通过 3-4 个最关键指标精准识别出最高影响的性能瓶颈，提出唯一优化方法，并制定具体的修改方案。所有分析与建议需严格基于数据，确保精准、可量化、可验证。

规则：

- 总回报 唯一的优化方法——最大期望加速比。
- 优先选择能够直接解决已测量瓶颈的更改（占用率限制、内存合并问题、共享内存银行冲突、寄存器压力、长/短流水线停顿、张量核心利用率不足等）。
- 保持字段简洁；避免列出替代方案、免责声明或通用建议。

输出格式 (JSON):

```
{
  "bottleneck": "<max 30 words>",
  "optimisation method": "<max 35 words>",
  "modification plan": "<max 35 words>"
}
```

目标显卡

GPU Name: {gpu_name}

Architecture: {gpu_arch}

Details:

{gpu_items}

PyTorch 参考

{python_code}

CUDA 候选者

{CUDA_CODE}

Nsight Compute 指标 (原文)

{NCU_METRICS}

请阅读所有内容并严格遵守规则。以指定格式返回 JSON。

A.3 从第二轮到第 N 轮的程序员提示

在获得裁判的反馈后，程序员将根据反馈对当前的核函数候选者进行修正或最优化。以下是用于修正和最优化的提示：

Prompt for Kernel Correction

你是一名资深的 CUDA 扩展开发者。你的任务是修复以下 Python 脚本中的编译或运行时错误。

输出规则 (严格)

1. Inside the block, follow exactly this order:
 1. Imports - torch, torch.nn, load_inline.
 2. source - triple-quoted CUDA string(s) (kernel + host wrapper).
 3. cpp_src - prototypes for all kernels you expose.
 4. One load_inline call per kernel group.

5. class ModelNew(nn.Module) - mirrors original inputs/outputs but calls your CUDA kernels.
2. Do NOT include testing code, if `__name__ == "__main__"`, or extra prose.

错误日志

{ERROR_LOG}

旧代码（只读）

{CUDA_CODE}

主要评论问题

{Problem}

输出部分（待生成）：

<your corrected code>

Prompt for Kernel Optimization

目标显卡

GPU Name: {gpu_name}

Architecture: {gpu_arch}

Details:

{gpu_items}

你是一名 CUDA 内核最优化专家。

分析所提供的架构并严格应用以下策略以生成改进的 CUDA 内核。

{CUDA_CODE}

最优化说明：

{optimization_suggestion}

目标

- Improve latency and throughput on the target GPU.
- Maintain correctness within $\text{atol}=1\text{e-}4$ or $\text{rtol}=1\text{e-}4$.

- Preserve the public Python API (same inputs/outputs, shapes, dtypes).

输出规则（严格）

1. Inside the block, follow exactly this order:

1. Imports - torch, torch.nn, load_inline.
2. source - triple-quoted CUDA string(s) (kernel + host wrapper).
3. cpp_src - prototypes for all kernels you expose.
4. One load_inline call per kernel group.
5. class ModelNew(nn.Module) - mirrors original inputs/outputs but calls your CUDA kernels.

2. Do NOT include testing code, if `__name__ == "__main__"`, or extra prose.

输出部分（待生成）:

```
# <your corrected code>
```

B NCU 指标详情

B.1 为何选择 NCU 子集指标？

我们发现，将大型模型暴露于完整的 NCU 指标集会导致模型过载，降低其最优化建议的准确率和稳定性，并影响 Judge 输出质量。我们通过以下具体案例研究来说明这一点。

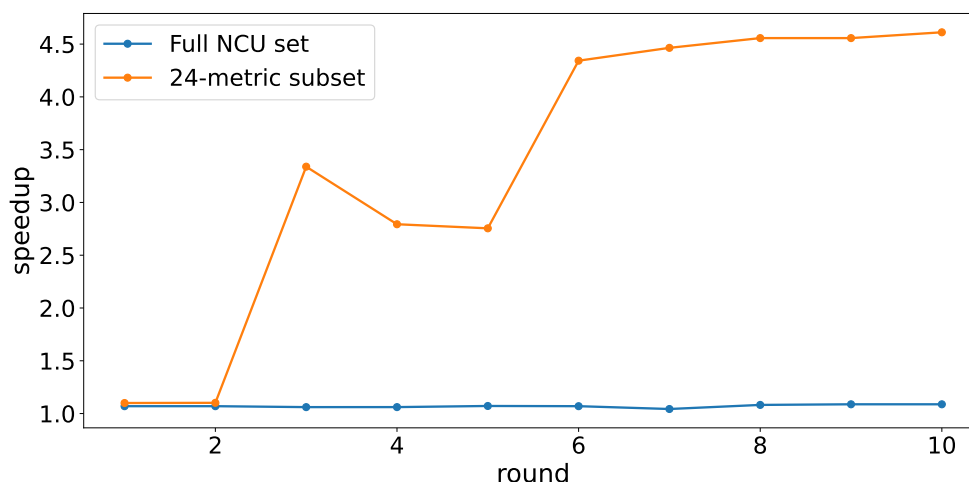


Figure 9: 针对二级任务 51 的评判智能体，全指标输入（蓝线）与指标子集输入（橙线）生成的内核性能对比。

如图 9 所示，我们在一个代表性任务上比较了两种配置。(i) 判定智能体在每轮中接收所有原

始的 NCU 指标，并向编码智能体提供反馈。(ii) 判定智能体仅接收我们方法筛选出的 24 个关键指标，并在每轮中仅捕捉 3-4 个关键指标以生成最优化建议。

在 10 轮迭代中，使用完整 NCU 指标的 Judge 智能体无法引导 Coder 智能体生成更优的内核，生成的内核性能几乎保持不变，尽管经历了多次交互。相比之下，当 Judge 智能体被限制使用子集指标时，生成内核的性能在各轮之间出现变化，并经常得到改善。这一过程最终带来了显著的加速效果。

此外，我们仔细审查了法官如何识别两种配置（全集与 24 个子集）中的瓶颈，并提出优化方案。我们选取了优化阶段中一位法官的回复，以更清晰地展示法官输出的差异：一种是使用全部 NCU 作为输入，另一种是使用部分 NCU 作为输入。

Response by using the full NCU metrics set

```
{ "bottleneck": "Long-scoreboard stalls from re-loading GEMM output out of DRAM (gemm_out round-trip dominates cycles).",  
  
  "optimisation method": "Fuse the Linear GEMM and the current epilogue into one FP16 tensor-core CUTLASS kernel that performs subtract, row-mean, GELU and residual add in-register, avoiding global-memory re-reads.",  
  
  "modification plan": "Write CUTLASS custom epilogue functor; use cublasLt or cutlass:: gemm_universal to compute GEMM+A, keep accumulators in registers, warp-reduce to mean, apply GELU, add x_orig, directly store final output; expose via new PyTorch extension." }
```

Response by using the 24 key NCU subset

```
{ "bottleneck": "Kernel is DRAM-bound (102.9 second pass reading original_x.",  
  
  "optimisation method": "Fuse the second loop: cache each thread's original_x elements in registers during diff accumulation, then add gelu_val and store, eliminating the extra B×F global read.",  
  
  "modification plan": "Replace two for-loops with one: load linear_out, subtract, and original_x together, keep original_x[16] in per-thread array, finish reduction, add gelu_val, write to output; ~4 MB traffic per batch, >30
```

基于这两个响应，我们发现使用完整 NCU 指标集的裁判倾向于错误识别真正的性能瓶颈。使用完整 NCU 指标集的裁判将瓶颈归因于重新加载 gemm_out，并建议采用一个单体式 CUTLASS 后处理（epilogue），在寄存器中执行行均值/GELU/残差计算。这一诊断与我们内核的访问模式不符，且由于跨块聚合的存在，对通用形状难以实现。相比之下，使用 24 个关键指标子集的裁判正确

识别出该内核为受 DRAM 限制的内核，其主要开销来自对 `x_orig` 的第二次遍历，并提出了一种单次遍历的重构方案：在第一次遍历时将 `x_orig` 缓存到寄存器中，经过 GELU 后写回，从而消除一次完整的 $B \times F$ 全局内存读取。此改动轻量、架构无关，且在各种情景下均能带来稳定加速（例如，每批量减少约 4 MB 的内存流量，在我们的设置中提升超过 30%）。

B.2 NCU 前 20 名指标示例

本节通过若干示例任务，报告了与运行时间相关性最强的前 20 个 Nsight Compute (NCU) 指标，按皮尔逊相关系数的绝对值排序。此处的运行时间指内核的执行时间。当相关系数为正时，较大的指标值通常意味着更长的执行时间；当相关系数为负时，较大的指标值通常意味着更短的执行时间。所有指标名称均遵循 NCU 中的原始名称。

Table 6: 任务-Conv2D：相关系数与运行时间（前 20 名）。

Metric Name	Correlation	Abs Correlation
<code>sm__cycles_active.avg</code>	1.000 000	1.000 000
<code>gpc__cycles_elapsed.max</code>	1.000 000	1.000 000
<code>launch__occupancy_limit_shared_mem</code>	0.945 507	0.945 507
<code>dram__bytes.sum.per_second</code>	-0.924 251	0.924 251
<code>gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed</code>	-0.924 155	0.924 155
<code>smsp__inst_executed.avg</code>	0.916 287	0.916 287
<code>smsp__inst_executed.sum</code>	0.916 287	0.916 287
<code>smsp__inst_issued.avg</code>	0.916 262	0.916 262
<code>smsp__inst_issued.sum</code>	0.916 262	0.916 262
<code>lts__t_sector_hit_rate.pct</code>	0.839 237	0.839 237
<code>smsp__sass_average_branch_targets_threads_uniform.pct</code>	0.810 334	0.810 334
<code>lts__throughput.avg.pct_of_peak_sustained_elapsed</code>	-0.787 261	0.787 261
<code>smsp__inst_executed_op_branch.sum</code>	0.746 483	0.746 483
<code>launch__grid_size</code>	0.745 917	0.745 917
<code>l1tex__t_sector_hit_rate.pct</code>	0.728 356	0.728 356
<code>gpc__cycles_elapsed.avg.per_second</code>	0.728 053	0.728 053
<code>dram__cycles_elapsed.avg.per_second</code>	0.665 784	0.665 784
<code>launch__waves_per_multiprocessor</code>	0.627 478	0.627 478
<code>launch__thread_count</code>	0.627 478	0.627 478
<code>launch__shared_mem_per_block_static</code>	-0.610 501	0.610 501

Table 7: 任务-SpMM：相关系数与运行时间（前 20 名）。

Metric Name	Correlation	Abs Correlation
<code>gpc__cycles_elapsed.max</code>	0.999 993	0.999 993
<code>sm__cycles_active.avg</code>	0.998 432	0.998 432
<code>gpu__compute_memory_request_throughput.avg.pct_...</code>	-0.967 284	0.967 284
<code>gpu__compute_memory_throughput.avg.pct_of_peak_...</code>	-0.964 455	0.964 455
<code>lts__t_sector_hit_rate.pct</code>	0.951 201	0.951 201
<code>dram__bytes.sum.per_second</code>	-0.926 134	0.926 134

Continued on next page

Metric Name	Correlation	Abs Correlation
gpu__dram_throughput.avg.pct_of_peak_sustained...	-0.925 856	0.925 856
lltex__throughput.avg.pct_of_peak_sustained_active	0.871 262	0.871 262
sm__inst_executed.avg.per_cycle_elapsed	-0.837 675	0.837 675
smsp__issue_inst0.avg.pct_of_peak_sustained_active	0.837 284	0.837 284
smsp__issue_active.avg.pct_of_peak_sustained...	-0.837 284	0.837 284
smsp__issue_active.avg.per_cycle_active	-0.837 283	0.837 283
sm__inst_issued.avg.per_cycle_active	-0.836 185	0.836 185
sm__inst_issued.avg.pct_of_peak_sustained_active	-0.836 185	0.836 185
sm__inst_executed.avg.per_cycle_active	-0.836 160	0.836 160
sm__instruction_throughput.avg.pct_of_peak_sust...	-0.806 478	0.806 478
smsp__average_warp_latency_per_inst_issued.ratio	0.802 793	0.802 793
smsp__average_warps_active_per_inst_executed.ratio	0.802 777	0.802 777
derived__smsp__inst_executed_op_branch_pct	-0.728 768	0.728 768
smsp__warps_eligible.avg.per_cycle_active	-0.630 772	0.630 772

B.3 24 个 NCU 指标的关键子集

下表列出了我们 NCU 关键子集中的确切 24 个度量，这是算法 1、2 的结果

Table 8: 24 米度量关键子集。

#	Metric Name
1	sm__cycles_active.avg
2	sm__warps_active.avg.pct_of_peak_sustained_active
3	launch__occupancy_limit_blocks
4	launch__occupancy_limit_registers
5	launch__occupancy_limit_shared_mem
6	launch__registers_per_thread
7	sm__inst_executed.sum
8	sm__inst_executed_pipe_fp32.avg.pct_of_peak_sustained_active
9	sm__inst_executed_pipe_tensor.avg.pct_of_peak_sustained_active
10	dram__bytes_read.sum
11	dram__bytes_write.sum
12	dram__throughput.avg.pct_of_peak_sustained_elapsed
13	dram__bytes.sum.per_second
14	gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed
15	lltex__t_sector_hit_rate.pct
16	lltex__throughput.avg.pct_of_peak_sustained_active
17	lts__t_sector_hit_rate.pct
18	lts__throughput.avg.pct_of_peak_sustained_active
19	smsp__warp_issue_stalled_memory_dependency_per_warp_active.pct
20	smsp__warp_issue_stalled_short_scoreboard_per_warp_active.pct
21	smsp__warp_issue_stalled_long_scoreboard_per_warp_active.pct
22	smsp__warp_issue_stalled_barrier_per_warp_active.pct
23	smsp__warp_issue_stalled_branch_resolving_per_warp_active.pct
24	smsp__sass_average_branch_targets_threads_uniform.pct

C CUDA-L1

在我们的复现过程中，我们发现 CUDA-L1 的作者仅发布了每个任务最终生成的内核。通过对这些案例的仔细研究，我们发现了几个有趣的结论。

首先，我们发现 CUDA-L1 倾向于强调 PyTorch 层面的最优化，而非生成和优化自定义的 CUDA 内核。这一模式在他们提供的案例中也表现为最常见的问题。尽管 CUDA-L1 报告了速度提升最大的前 10 个案例，但我们的审查显示，这十个最终解决方案中有九个并未使用自定义 CUDA 内核；相反，它们高度依赖官方 PyTorch 实现。以下是他们结果的几个案例。

这是其 *KernelBench* 任务按 *RL-CUDA1* 加速效果排序（前 10 名）中排名第一的条目：第 2 级任务 83，报告的 $120.3\times$ 加速比。该任务不包含任何 CUDA 内核。

Level 2 Task 83 —Reported $120.3\times$ Speedup

```
import torch
import torch.nn as nn

class ModelNew(nn.Module):
    """
    Optimized implementation of a model that performs a 3D convolution,
    applies Group Normalization, minimum, clamp, and dropout.

    Args:
        in_channels (int): Number of input channels
        out_channels (int): Number of output channels
        kernel_size (int or tuple): Size of the convolving kernel
        groups (int): Number of groups for GroupNorm
        min_value (float): Minimum value for clamp operation
        max_value (float): Maximum value for clamp operation
        dropout_p (float): Dropout probability
    """
    def __init__(self, in_channels, out_channels, kernel_size, groups, min_value,
max_value, dropout_p):
        super(ModelNew, self).__init__()
        # Store the original layers for parameter compatibility
        self.conv = nn.Conv3d(in_channels, out_channels, kernel_size)
        self.norm = nn.GroupNorm(groups, out_channels)
        self.dropout = nn.Dropout(dropout_p)
        self.min_value = min_value
```

```

self.max_value = max_value
self.dropout_p = dropout_p

# Pre-compute convolution parameters for output shape calculation
if isinstance(kernel_size, int):
    self.kernel_size = (kernel_size, kernel_size, kernel_size)
else:
    self.kernel_size = kernel_size

self.stride = self.conv.stride
self.padding = self.conv.padding
self.dilation = self.conv.dilation

def forward(self, x):
    x = self.conv(x)
    x = self.norm(x)
    x = torch.minimum(x, torch.tensor(self.min_value, device=x.device))
    x = torch.clamp(x, min=self.min_value, max=self.max_value)
    x = self.dropout(x)
    return x

# Keep ALL hyperparameters EXACTLY as shown in the reference implementation
batch_size = 128
in_channels = 3
out_channels = 16
depth, height, width = 16, 32, 32
kernel_size = 3
groups = 8
min_value = 0.0
max_value = 1.0
dropout_p = 0.2

def get_inputs():
    return [torch.randn(batch_size, in_channels, depth, height, width)]

def get_init_inputs():

```

```
return [in_channels, out_channels, kernel_size, groups, min_value, max_value,
        dropout_p]
```

第二秩的情况是第 1 级任务 12（对角矩阵的矩阵乘法），报告的加速比为 64.4 倍，且其中不包含 CUDA 内核：

Level 1 Task 12 —Reported 64.4× Speedup

```
# diag_mm_compare.py
import time
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

# -----
# Reference implementation
# -----
class Model(nn.Module):
    """
    Simple model that performs a matrix multiplication of a diagonal matrix with another
    matrix.
    C = diag(A) * B
    """
    def __init__(self):
        super(Model, self).__init__()

    def forward(self, A, B):
        """
        Args:
            A (torch.Tensor): 1D tensor, diagonal entries. Shape: (N,)
            B (torch.Tensor): 2D tensor. Shape: (N, M)
        Returns:
            torch.Tensor: (N, M)
        """
        return torch.diag(A) @ B
```

```

# -----
# Optimized implementation
# -----
class ModelNew(nn.Module):
    """
    Optimized model that performs a matrix multiplication of a diagonal matrix with another
    matrix.
    C = diag(A) * B
    """
    def __init__(self):
        super(ModelNew, self).__init__()

    def forward(self, A, B):
        """
        Args:
            A (torch.Tensor): 1D tensor, diagonal entries. Shape: (N,)
            B (torch.Tensor): 2D tensor. Shape: (N, M)
        Returns:
            torch.Tensor: (N, M)
        """
        # Equivalent to torch.diag(A) @ B, but avoids forming the full diagonal matrix
        return B * A.unsqueeze(1)

# -----
# Hyperparameters & inputs
# -----
M = 4096
N = 4096

def get_inputs(device=None, dtype=torch.float32):
    A = torch.randn(N, device=device, dtype=dtype)
    B = torch.randn(N, M, device=device, dtype=dtype)
    return [A, B]

def get_init_inputs():

```

```
return [] # No special initialization inputs needed
```

此外，我们观察到许多报告的加速比实际上等于 1（集中在 1.00 附近，通常在 $\pm 5\%$ 范围内）。进一步检查发现，在这些情况下，当自定义内核编译失败时，系统会回退到原始的 PyTorch 操作，这自然不会产生可测量的加速。

例如，以下是 CUDA-L1 生成的 KernelBench Level-1 Task 3 最终解决方案中的 forward 方法。该代码来自 CUDA-L1 官方 Github。我们观察到，该方法首先尝试调用一个自定义 CUDA 内核；然而，一旦出现编译失败或异常，便会立即回退到 `torch.bmm(A, B)`。关键在于，`torch.bmm(A, B)` 正是本任务要求被自定义内核替代的操作符，这意味着回退机制破坏了任务的客观目标。这解释了为何报告的加速比仅为 $1.006\times$ 。

Level 1 Task 3 —Reported $1.006\times$ Speedup

```
def forward(self, A: torch.Tensor, B: torch.Tensor) -> torch.Tensor:
    """
    Performs batched matrix multiplication.

    Args:
        A: Input tensor of shape (batch_size, m, k).
        B: Input tensor of shape (batch_size, k, n).

    Returns:
        C: Output tensor of shape (batch_size, m, n).
    """
    # Fall back to torch.bmm if CUDA module failed to load
    if ModelNew._cuda_module is None:
        return torch.bmm(A, B)

    # Check if inputs are on CUDA
    if not A.is_cuda or not B.is_cuda:
        A = A.cuda() if not A.is_cuda else A
        B = B.cuda() if not B.is_cuda else B

    # Ensure inputs are contiguous and float32
    A = A.contiguous().float()
    B = B.contiguous().float()

    # Use custom CUDA kernel
```

```

try:
    result = ModelNew._cuda_module.batched_matmul(A, B)
    if not A.is_cuda:
        result = result.cpu()
    return result
except Exception as e:
    print(f"Error in custom kernel: {e}, falling back to torch.bmm")
    return torch.bmm(A, B)

```

D 基准详情

D.1 内核基准测试

KernelBench 是一个标准化的基准，旨在评估大语言模型（LLMs）在 CUDA 内核生成与最优化方面的能力。它包含四个难度递增级别的共 270 个任务，其中 Level 1–3（总计 250 个任务）通常用于评估。每个任务均提供 PyTorch 的参考实现以及固定的输入-输出规范，支持自动化正确性与性能验证。

- **一级（基本运算符）**：包含矩阵乘法、逐元素运算和归约等简单、低级别的运算符。这些任务主要测试生成功能正确的 CUDA 内核的能力。
- **第二级（复合操作）**：涉及多步算子组合，要求模型组合多个 CUDA 原语并高效管理中间内存。这些任务测试模型进行更复杂代码合成的能力。
- **第三级（端到端模型）**：包含从完整的神经网络结构（如 AlexNet、VGG 和残差网络组件）中衍生出的复杂内核。这些任务评估在真实的深度学习负载下生成高效、大规模内核的能力。
- **Level 4（可选）**：完整的基准还定义了一个高级别，包含额外的研究导向任务，但由于其复杂性以及缺乏标准化的评估设置，这一级别较少被采用。

KernelBench 在最近关于基于大语言模型（LLM）的代码生成工作中已成为广泛使用的基准 [??]，因为它提供了一个受控且可复现的环境来衡量两个方面：正确性（与 PyTorch 的功能等价性）和效率（相对于 PyTorch 的执行速度）。在我们的研究中，我们采用所有 Level 1–3 的任务，遵循先前的工作，以确保与各类基准之间的公平比较。

D.2 我们分层随机子集 $\mathcal{D}^*$

尽管我们的主要评估是在完整的 KernelBench Level 1–3 基准（共 250 个任务）上进行的，我们还额外构建了一个分层子集 $\mathcal{D}^*$ ，以支持详细的分析和公平比较，同时降低实验中的计算代价。

$\mathcal{D}^*$ 的构建遵循两个原则：(1) **不同难度层级的覆盖**。由于 KernelBench 按任务复杂度递增进行分层（第 1 层：单操作符任务，第 2 层：多步融合操作符，第 3 层：完整网络组件），我们确保采样

的子集保持难度的相对分布。(2) **任务类型的多样性**。在每一层内，我们对不同操作符类别（例如，逐元素操作、归约操作、卷积、融合块）进行均匀采样，以保证子集能够代表整体基准的多样性。

具体而言，我们在每个层级上执行固定比例为 10% 的分层随机采样，结果得到第 1 级的 10 个任务、第 2 级的 10 个任务以及第 3 级的 5 个任务，总计 25 个任务。为了保证可复现性， $\mathcal{D}^*$ 中包含的确切任务 ID 为：

- **一级 (10 题)：** 13, 10, 16, 29, 35, 72, 7, 89, 93, 34
- **第二级 (10 个任务)：** 17, 19, 40, 3, 13, 21, 38, 28, 26, 34
- **第三级 (5 个任务)：** 5, 18, 32, 41, 21